

GenAi platform: claude gemini:ccessed Google Gemini through the Gemini web interface and Claude through the Claude web interface.

Collaboration: ChatGPT explanation of some of the concepts of C++ and the vocabulary of cpp Prompt

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.

- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:
Subject:
Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails

```
// CEO Email Prioritizer - object-oriented, hand-built list-based MaxHeap.  
// Build: g++ -std=c++11 -o ceo_email ceo_email.cpp  
// Run: ./ceo_email commands.txt    (or pipe commands via stdin)
```

```
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <string>  
#include <vector>  
#include <cstdlib>
```

```
// ----- Email
```

```
class Email {  
private:  
    std::string sender;  
    std::string subject;  
    std::string dateStr; // MM-DD-YYYY as given  
    int categoryRank;    // higher = read sooner  
    int dateKey;         // YYYYMMDD for comparison  
    long sequence;       // arrival order, breaks exact ties (earlier first)  
  
    static int rankOf(const std::string& cat) {  
        if (cat == "Boss")        return 5;  
        if (cat == "Subordinate")  return 4;  
        if (cat == "Peer")        return 3;  
        if (cat == "ImportantPerson") return 2;  
        return 1;                 // OtherPerson  
    }  
  
    static int keyOf(const std::string& d) {  
        int mm = std::atoi(d.substr(0, 2).c_str());  
        int dd = std::atoi(d.substr(3, 2).c_str());  
        int yy = std::atoi(d.substr(6, 4).c_str());  
        return yy * 10000 + mm * 100 + dd;  
    }  
}
```

```

public:
    Email() : categoryRank(0), dateKey(0), sequence(0) {}
    Email(const std::string& s, const std::string& subj,
          const std::string& d, long seq)
        : sender(s), subject(subj), dateStr(d),
          categoryRank(rankOf(s)), dateKey(keyOf(d)), sequence(seq) {}

    // true if this email should be read BEFORE other
    bool hasHigherPriorityThan(const Email& other) const {
        if (categoryRank != other.categoryRank)
            return categoryRank > other.categoryRank;
        if (dateKey != other.dateKey)
            return dateKey > other.dateKey;    // newest first
        return sequence < other.sequence;      // stable for same date
    }

    void display() const {
        std::cout << "Sender: " << sender << "\n"
                   << "Subject: " << subject << "\n"
                   << "Date: " << dateStr << "\n";
    }
};

// ----- MaxHeap
// List (array) based binary max-heap, written from scratch.
class MaxHeap {
private:
    Email* data;
    int capacity;
    int count;

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i) { return 2 * i + 1; }
    static int right(int i) { return 2 * i + 2; }

    void swapAt(int a, int b) {
        Email t = data[a];
        data[a] = data[b];
        data[b] = t;
    }

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];

```

```

    for (int i = 0; i < count; i++) bigger[i] = data[i];
    delete[] data;
    data = bigger;
    capacity = newCap;
}

```

```

void siftUp(int i) {
    while (i > 0 && data[i].hasHigherPriorityThan(data[parent(i)]) {
        swapAt(i, parent(i));
        i = parent(i);
    }
}

```

```

void siftDown(int i) {
    while (true) {
        int best = i, l = left(i), r = right(i);
        if (l < count && data[l].hasHigherPriorityThan(data[best])) best = l;
        if (r < count && data[r].hasHigherPriorityThan(data[best])) best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}

```

public:

```

MaxHeap() : data(new Email[16]), capacity(16), count(0) {}
~MaxHeap() { delete[] data; }

```

```

// non-copyable (owns raw array)
MaxHeap(const MaxHeap&) = delete;
MaxHeap& operator=(const MaxHeap&) = delete;

```

```

void insert(const Email& e) {
    if (count == capacity) grow();
    data[count] = e;
    siftUp(count);
    count++;
}

```

```

bool isEmpty() const { return count == 0; }
int size() const { return count; }

```

```

const Email& peekMax() const { return data[0]; }

```

```

Email extractMax() {
    Email top = data[0];
    count--;
    if (count > 0) {
        data[0] = data[count];
        siftDown(0);
    }
    return top;
}
};

```

// ----- CEO Inbox

```

class CEOInbox {
private:
    MaxHeap heap;
    long nextSeq;

public:
    CEOInbox() : nextSeq(0) {}

    void receive(const std::string& sender, const std::string& subject,
                const std::string& date) {
        heap.insert(Email(sender, subject, date, nextSeq++));
    }

    void next() const {
        if (heap.isEmpty()) {
            std::cout << "No unread emails.\n";
            return;
        }
        heap.peekMax().display();
    }

    void read() {
        if (heap.isEmpty()) {
            std::cout << "No unread emails.\n";
            return;
        }
        heap.extractMax(); // CEO has read it and dealt with it
    }

    void count() const {
        std::cout << "Unread emails: " << heap.size() << "\n";
    }
}

```

```

};

// ----- Command parser
class CommandProcessor {
private:
    CEOInbox& inbox;

    static std::string trim(const std::string& s) {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == std::string::npos) return "";
        size_t b = s.find_last_not_of(" \t\r\n");
        return s.substr(a, b - a + 1);
    }

    static std::vector<std::string> splitCommas(const std::string& s) {
        std::vector<std::string> parts;
        std::string cur;
        std::stringstream ss(s);
        while (std::getline(ss, cur, ',')) parts.push_back(trim(cur));
        return parts;
    }

public:
    explicit CommandProcessor(CEOInbox& in) : inbox(in) {}

    void process(const std::string& rawLine) {
        std::string line = trim(rawLine);
        if (line.empty()) return;

        if (line.compare(0, 6, "EMAIL ") == 0) {
            std::vector<std::string> f = splitCommas(line.substr(6));
            if (f.size() != 3) {
                std::cerr << "Malformed EMAIL command: " << line << "\n";
                return;
            }
            inbox.receive(f[0], f[1], f[2]);
        } else if (line == "NEXT") {
            inbox.next();
        } else if (line == "READ") {
            inbox.read();
        } else if (line == "COUNT") {
            inbox.count();
        } else {
            std::cerr << "Unknown command: " << line << "\n";
        }
    }
};

```

```

    }
}
};

// ----- main
int main(int argc, char* argv[]) {
    CEOInbox inbox;
    CommandProcessor processor(inbox);

    std::istream* in = &std::cin;
    std::ifstream file;
    if (argc > 1) {
        file.open(argv[1]);
        if (!file) {
            std::cerr << "Cannot open " << argv[1] << "\n";
            return 1;
        }
        in = &file;
    }

    std::string line;
    while (std::getline(*in, line)) processor.process(line);
    return 0;
}

```

The Gemini version used several classes, including

Datakey

priority rank

Email

Max heap ceoinbox

Stringtokenizer

Command processor

The claude version also used an object-oriented design with

Email max heap

Ceoinbox

Command processor

Both of the programs got the required email rules they first prioritized emails according to the sender and used the date to determine the priority when two emails have the same sender level

Analysis of the two programs

Gemini :

correctness

The Gemini program correctly applied the function and used a max heap to organize the emails according to the required priority: the level of importance of the people and the date

The main function is correct; just the output format has a little problem, but it will be fixed very fast

Execution time

The Gemini program uses an array-based max heap, and here are the time complexities

Inserting an email: $O(\log n)$

Getting the next email is $O(1)$

Reading or removing the highest priority email is $O(\log n)$

Counting unread emails(count) $O(1)$

Sift up and sift down are also $O(\log n)$

The space complexity

The Gemini program uses a dynamically allocated array to store the emails in the max heap, and it is $O(N)$

The program also creates individual objects of email date key and priority rank; they won't affect the space complexity, but compared with Claude, it has more classes and variables

For the maintainability

The Gemini one has a better object-oriented structure, and it separates different responsibilities into different classes

The data key prioritizes the rank of the email in the inbox, which makes the code easier to understand and it's easier to do perfective maintenance because each class and function is clearly separated. The only thing is that we have to add comments to the code

Claude

The Claude program also correctly implemented the main requirements of the assignment, and it has applied the function and used a max heap to organize the emails according to the required priority: the level of importance of the people and the date

The same problems as Gemini: the form of the output has tiny differences from what we expect, but it can be easily fixed

The execution time

The Claude program also uses an array-based max heap and the same as the gemini

Inserting an email: $O(\log n)$

Getting the next email is $O(1)$

Reading or removing the highest priority email is $O(\log n)$

Counting unread emails(count) $O(1)$

Sift up and sift down are also $O(\log n)$

The space complexity

The Claude program stores emails in a dynamically allocated array used by the MaxHeap. With n emails, the program requires $O(n)$ space.

Which probably use less class than the gemini

The maintainability

The Claude program uses the object oriented design and separates the major parts of the program into email maxheap ceoinbox and command processor

But is less clearly structure than the gemini because, under the email, they have datekey priority rank which can make corrective maintenance, adaptive maintenance, and preventive maintenance

Compared

They are mostly the same level of complexity of time and correctness; maybe Claude has slightly better space complexity, but for maintenance, Gemini is better, so i choose Gemini for improving code

Improvement

Correctness the gemini

For example, the original Gemini program displayed the unread email count as:

Unread emails: 6

the required output format is: There are 6 emails to read.

I changed the displayCount() function so that it produces the exact output required by the assignment.

I also found that the original NEXT operation displayed the email information without first printing: Next email:

I modified the displayNextEmail() function to print this line before displaying the sender, subject, and date.

The execution time

So the program already uses the max heap to store and prioritize emails this is a good data structure, so i don't think it would need more improvement

Space complexity

I did not make the change to the data structure because the max heap is the best data structure in my knowledge

Maintainability :

So i improve the code organization and added clever comments that explain the purpose of the class, and ChatGPT probably helped me understand some of the concepts but all of the comments are write by myself